

《具身大模型理论》实验手册

实验 1：大模型搭建与训练

实验说明

本实验手册环境以 Ubuntu、CUDA、Python 3.10 为参考。本手册针对 Windows 10/11 原生命令行做了课堂适配；如果本机没有 NVIDIA 显卡，也可以完成环境检查和代码阅读，但完整训练会非常慢。

本实验所用大模型源代码来自一个开源的超小型语言模型项目 MiniMind，旨在帮助开发者以极低成本从零开始训练自己的语言模型。其最小版本仅需 25.8M 参数，体积仅为 GPT-3 的 1/7000，适合在普通个人 GPU 上快速训练和运行。

本实验报告主要有两个目的：

1. 学生可按步骤在自己电脑实现大模型的搭建；
2. 学生可学习并理解实验步骤，在服务器主机上进行验证和使用。

实验主线：认识电脑硬件 → 搭建环境 → 下载项目和数据 → 预训练 → SFT 微调 → 推理对比

推荐终端：Anaconda Prompt (Windows 命令行)

手册版本：《具身大模型理论》实验手册 V1.0，2026-05-16

0. 手册要点

序号	实验环节	学生要完成的结果
1	实验目标与流程	知道本次实验要训练什么、观察什么
2	Windows 基础与显卡检查	会用 <code>nvidia-smi</code> 判断电脑是否能用 NVIDIA GPU
3	创建 Python 环境	会创建并激活 <code>conda</code> 环境
4	安装 PyTorch 与验证 CUDA	能判断 <code>torch.cuda.is_available()</code> 是否为 <code>True</code>
5	下载 MiniMind 项目与依赖	能进入项目目录并安装 <code>requirements.txt</code>
6	准备数据与认识数据格式	能看到 <code>pretrain_t2t_mini</code> 和 <code>sft_t2t_mini</code> 的区别
7	预训练实验	能启动 <code>train_pretrain.py</code> 并保存/测试 <code>pretrain</code> 权重
8	SFT 微调实验	能启动 <code>train_full_sft.py</code> 并测试 <code>full_sft</code> 权重
9	实验记录与问题排查	能提交截图、结果对比和实验反思

1. 实验目标与课程主线

本实验不要求学生一开始就理解所有代码细节，而是先把“大模型训练的一条完整流水线”跑通。完成后，学生应能解释：为什么要检查显卡，为什么文本要经过 `Tokenizer`，预训练和 SFT 微调分别解决什么问题。

- 目标 1：认识 Windows 电脑上的 CPU、内存、显卡、驱动和 CUDA 可见性。
- 目标 2：学会使用 Anaconda Prompt 执行基本命令。
- 目标 3：搭建 MiniMind 训练环境，并完成 PyTorch CUDA 检查。
- 目标 4：使用 MiniMind 官方推荐的 mini 数据，跑通 Pretrain 和 Full SFT 两个关键阶段。
- 目标 5：对比预训练模型和微调模型的回答差异，理解“会续写”和“会按指令回答”的区别。

阶段	要解决的问题	学生观察重点
环境检查	电脑是否具备 GPU 训练条件	<code>nvidia-smi</code> 、显存、 <code>torch.cuda.is_available()</code>
数据准备	训练文本从哪里来、长什么样	<code>pretrain</code> 是纯文本，SFT 是多轮对话
Tokenizer	文字如何变成模型能处理的数字序列	文本 → token → id → embedding
预训练	让模型先学习语言规律和基础知识	loss 是否下降，回答是否像文本续写
SFT 微调	让模型学习指令跟随和助手式回答	回答格式、身份口径、任务完成度是否提升

2. 上课前准备：请统一使用这些软件

本手册默认使用 Windows 10/11 + Anaconda Prompt。不要把项目放在中文路径、带空格的路径或 OneDrive 同步目录中，建议统一放在 D:\AI\minimind。

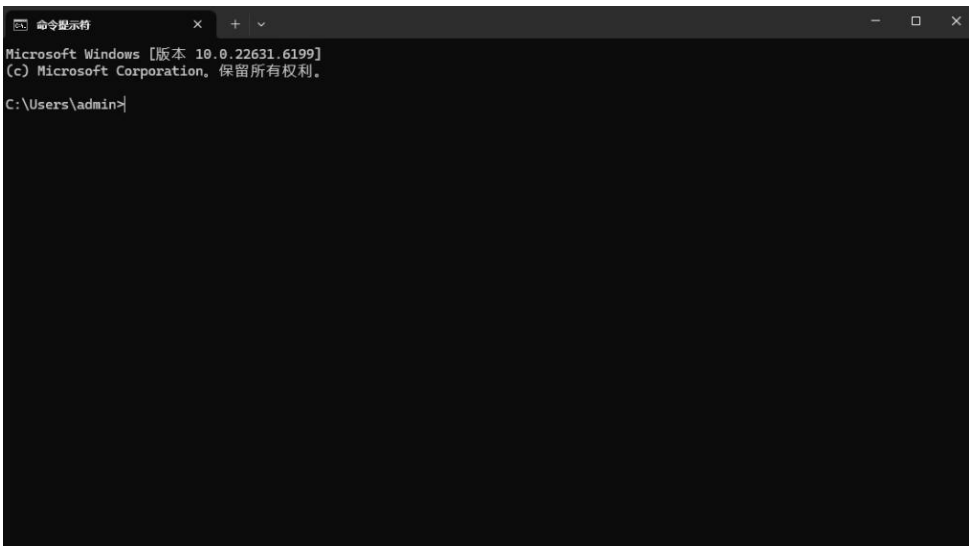
类别	推荐软件/条件	说明
操作系统	Windows 10 或 Windows 11	建议使用 64 位系统。
终端	Anaconda Prompt	开始菜单搜索 Anaconda Prompt，后续命令默认在这里执行。
Python	Python 3.10	与 MiniMind 官方参考 Python 3.10.x 保持接近。
包管理	Miniconda 或 Anaconda	用于创建独立实验环境，避免污染系统 Python。
代码工具	Git for Windows	用于克隆 MiniMind 仓库；没有 Git 时可用 ZIP 备选方案。
显卡	NVIDIA GPU 优先	没有 NVIDIA GPU 也能读代码和做部分 CPU 测试，但训练非常慢。
磁盘空间	至少 10GB，建议 30GB+	mini 数据约数 GB，加上依赖和权重需要额外空间。

3. 第一步：认识自己的 Windows 电脑有没有 NVIDIA 显卡

很多学生第一次做深度学习实验时，会把“有显卡”和“能用 CUDA 训练”混在一起。Windows 任务管理器里看到显卡，不代表 PyTorch 一定能调用它；必须同时检查 NVIDIA 驱动和 PyTorch 的 CUDA 可见性。

3.1 打开 cmd 窗口

1. 点击 Windows 开始菜单。
2. 搜索 cmd。
3. 打开后会看到一个黑色命令行窗口。
4. 以后本手册中的命令，默认都在这个类型的窗口中输入。



3.2 查看 NVIDIA 显卡：nvidia-smi

在 cmd 输入下面命令，然后按回车：

```
nvidia-smi
```

如果命令成功，会看到类似表格输出。重点看四个位置：

位置	含义	怎么判断
GPU Name	显卡型号	例如 RTX 3060、RTX 3090、RTX 4090 等。
Driver Version	NVIDIA 驱动版本	驱动太旧可能导致 CUDA/PyTorch 不可用。
CUDA Version	当前驱动支持的最高 CUDA 运行版本	这是驱动能力，不等于你已经安装了完整 CUDA Toolkit。
Memory-Usage	显存使用情况	训练时显存不足会报 CUDA out of memory。

```
C:\Users\admin>nvidia-smi
Sun May 10 20:23:29 2026

+-----+
| NVIDIA-SMI 591.86                Driver Version: 591.86      CUDA Version: 13.1     |
+-----+-----+
| GPU   Name                Driver-Model  Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf          Pwr:Usage/Cap       Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0   NVIDIA GeForce RTX 5090    WDDM        00000000:A8:00:0  On   |      1%      Default |
| 0%   40C    P0              61W /  575W      1131MiB / 32607MiB |                      |
+-----+-----+

+-----+
| Processes:                        |
| GPU   GI    CI          PID    Type   Process name                      GPU Memory |
| ID    ID    ID                         Usage   |
+-----+-----+
| 0     N/A   N/A               5612   C+G    ...xyewy\ShellExperienceHost.exe   N/A       |
| 0     N/A   N/A               6364   C+G    ...it)\bin\win64\MATLABWebUI.exe   N/A       |
| 0     N/A   N/A               7532   C+G    ...8wekyb3d8bbwe\M365Copilot.exe   N/A       |
| 0     N/A   N/A               8632   C+G    ...b(64bit)\bin\win64\MATLAB.exe    N/A       |
| 0     N/A   N/A              13080   C+G    ...y\StartMenuExperienceHost.exe    N/A       |
| 0     N/A   N/A              18248   C+G    ...IA App\CEF\NVIDIA Overlay.exe     N/A       |
| 0     N/A   N/A              19972   C+G    ...IA App\CEF\NVIDIA Overlay.exe     N/A       |
| 0     N/A   N/A              21012   C+G    ...bhw\PhoneExperienceHost.exe      N/A       |
+-----+-----+
```

如果 nvidia-smi 报错：常见原因：电脑没有 NVIDIA 显卡；NVIDIA 驱动未安装；驱动安装后没有重启；命令行没有识别到驱动路径。先用任务管理器或设备管理器确认显卡型号，再更新 NVIDIA 驱动。

3.3 查看 Windows 识别到的所有显卡

有些电脑同时有 Intel/AMD 核显和 NVIDIA 独显。可以用下面命令查看 Windows 识别到的显卡名称：

```
wmic path win32_VideoController get name
```

```
C:\Users\admin>wmic path win32_VideoController get name
Name
GameViewer Virtual Display Adapter
Parsec Virtual Display Adapter
Microsoft 基本显示适配器
NVIDIA GeForce RTX 5090
```

4. 第二步：创建 MiniMind 专用 Python 环境

为了避免不同课程或项目之间的 Python 包互相影响，本实验单独创建一个名为 minimind 的 conda 环境。

（注意，此处代码学生无需操作，服务器已预先装好 minimind 环境，这里仅作演示）

```
conda -version

conda create -n minimind python=3.10 -y

conda activate minimind

python -m pip install --upgrade pip
```

每次重新打开终端都要激活环境：以后只要重新打开终端，都先执行 `conda activate minimind`，再进入项目目录。看到命令行前面出现 `(minimind)`，说明环境已激活。

如图：`(minimind) C:\Users\admin>`

5. 第三步：（如环境初始安装 PyTorch）安装 PyTorch，并确认 CUDA 能不能用

MiniMind 使用 PyTorch 训练模型。PyTorch 是否能调用显卡，是本实验能否顺利训练的关键。

5.1 推荐安装方式

请先打开 PyTorch 官方安装页面，选择 Windows、Pip、Python，然后按自己电脑情况选择 CUDA 版本或 CPU。

PyTorch 官方安装页面：<https://pytorch.org/get-started/locally/>

不要盲目安装 CUDA Toolkit：对多数学生而言，使用 PyTorch 官方提供的 CUDA 版 pip wheel 即可。
`nvidia-smi` 主要检查驱动是否存在；只有需要 `nvcc` 编译 CUDA 程序时，才通常需要额外安装完整 CUDA Toolkit。

5.2 验证 PyTorch 是否安装成功

```
python -c "import torch; print('torch 版本:', torch.__version__); print('CUDA 是否可用:', torch.cuda.is_available()); print('torch 使用的 CUDA 版本:', torch.version.cuda); print('显卡名称:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'No CUDA')"
```

判断标准:

- 如果输出 CUDA 是否可用: True, 说明 PyTorch 已能调用 NVIDIA GPU。
- 如果输出 CUDA 是否可用: False, 说明后续训练会走 CPU, 速度会非常慢; 应先检查驱动、PyTorch CUDA 版本和环境是否激活。
- torch.version.cuda 显示的是 PyTorch 包所带 CUDA 运行时版本, 不一定和 nvidia-smi 显示完全相同。



```
(minimind) C:\Users\admin>python -c "import torch; print('torch版本:', torch.__version__); print('CUDA是否可用:', torch.cuda.is_available()); print('torch使用的CUDA版本:', torch.version.cuda); print('显卡名称:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'No CUDA')"
```

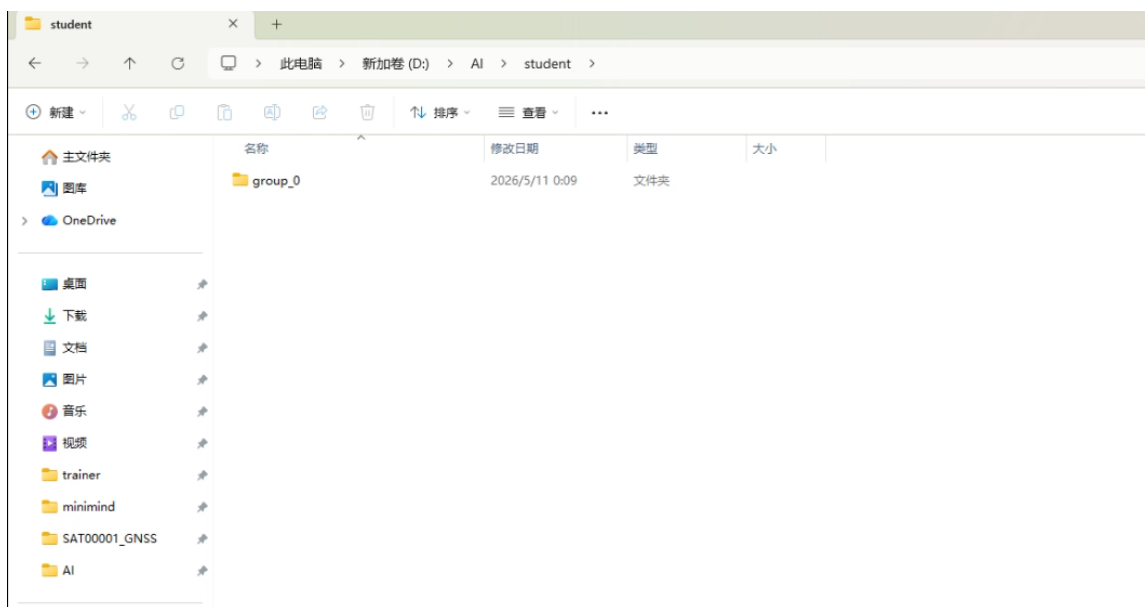
torch版本: 2.11.0+cu130
CUDA是否可用: True
torch使用的CUDA版本: 13.0
显卡名称: NVIDIA GeForce RTX 5090

6. 进入服务器操作: 创建小组文件夹

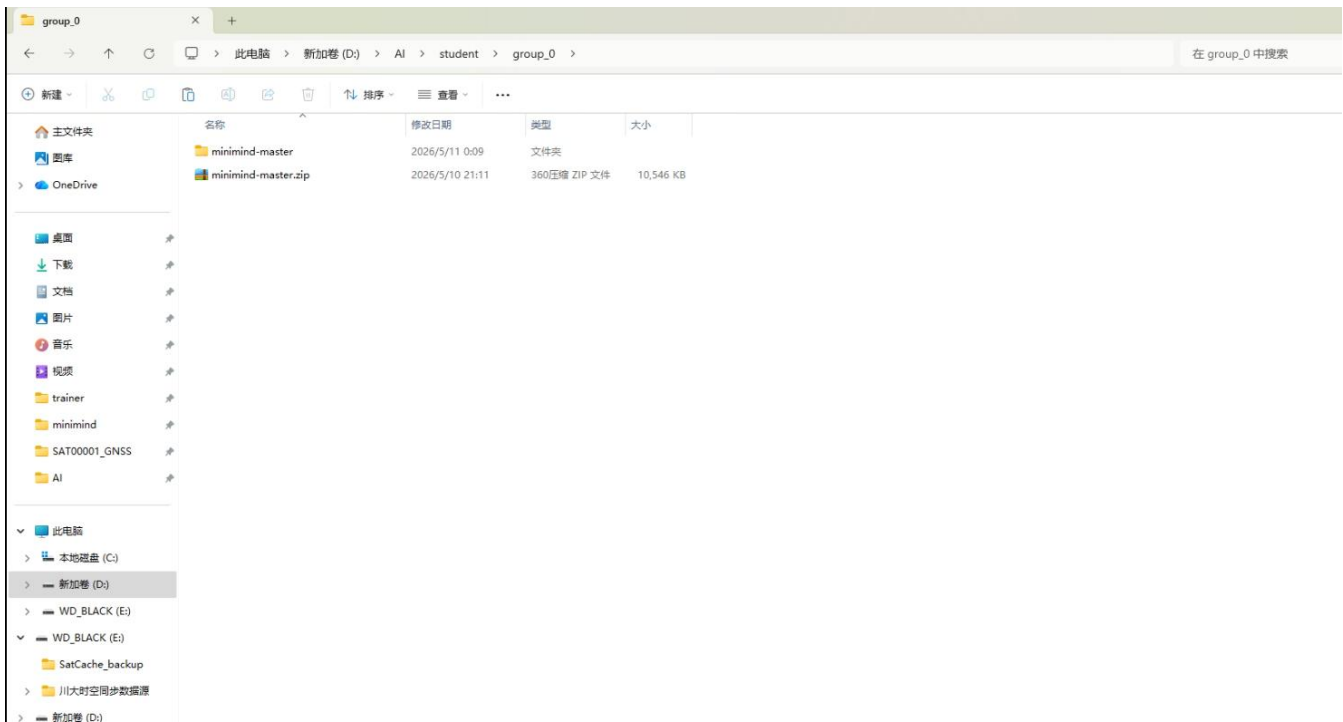
MiniMind 是一个从 0 实现大模型训练链路的开源项目。官方快速开始使用 git clone 下载仓库, 并通过 requirements.txt 安装依赖。Windows 下建议把项目放在 D:\AI。

6.1 小组预备操作

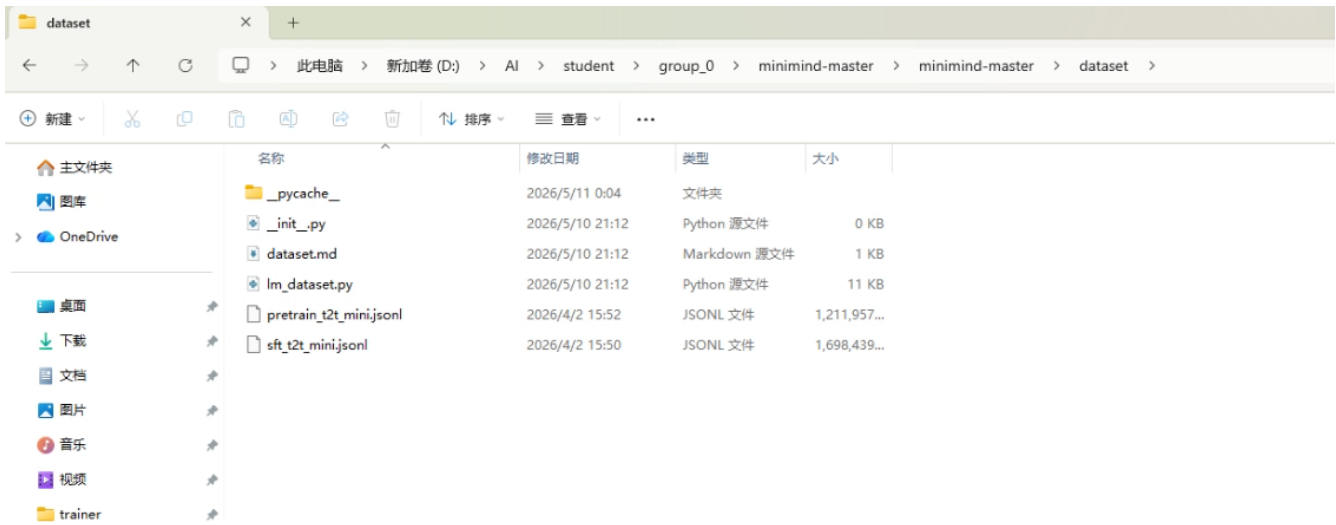
在 D:\AI\student 文件夹下创建自己小组的文件夹, 以下以 0 号小组为例: 创建 group_0 文件夹



复制 D:\AI 文件夹下的 minimind-master 压缩包到 group_0 文件夹并解压:



将 D:\AI\训练数据 文件夹下的 pretrain_t2t_mini.jsonl 和 sft_t2t_mini.jsonl 移动到 D:\AI\student\group_0\minimind-master\minimind-master\dataset 文件夹中，如下图所示



查看数据集：打开 pretrain_t2t_mini.jsonl 和 sft_t2t_mini.jsonl 观察数据集格式。
查看预训练和微调代码：打开 D:\AI\student\group_0\minimind-master\minimind-master\trainer 目录下的 train_pretrain.py 和 train_full_sft.py。

6.2 补充理解：Tokenizer 在这里做什么

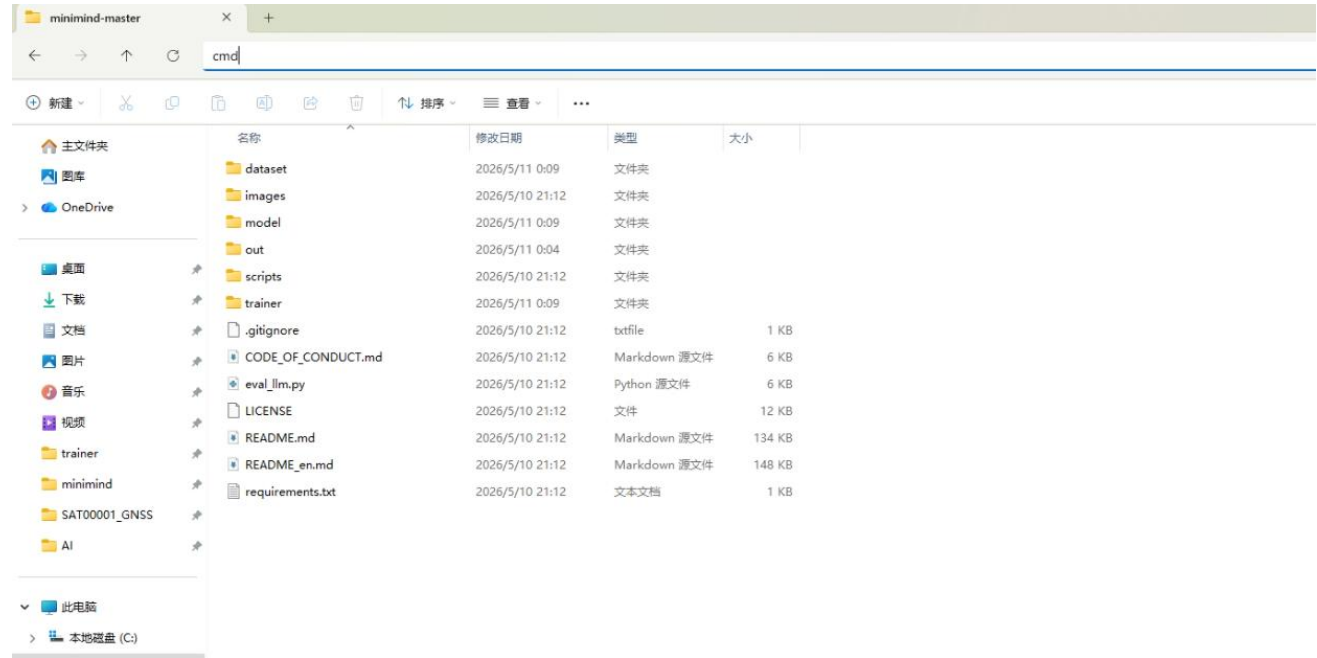
文本不能直接输入神经网络。Tokenizer 会把文字切成 token，再把 token 映射成数字编号，最后通过 embedding 变成向量。模型真正看到的是数字序列，不是人眼看到的中文句子。

步骤	示例	说明
原始文本	为什么天空是蓝色的？	人能直接读懂的文字。
Token	为 / 什么 / 天空 / 是 / 蓝色 / 的 / ？	实际切分方式由 tokenizer 决定，这里只是示意。
ID	101, 234, 567, ...	每个 token 对应词表里的编号。
Embedding	向量矩阵	模型用向量进行计算。

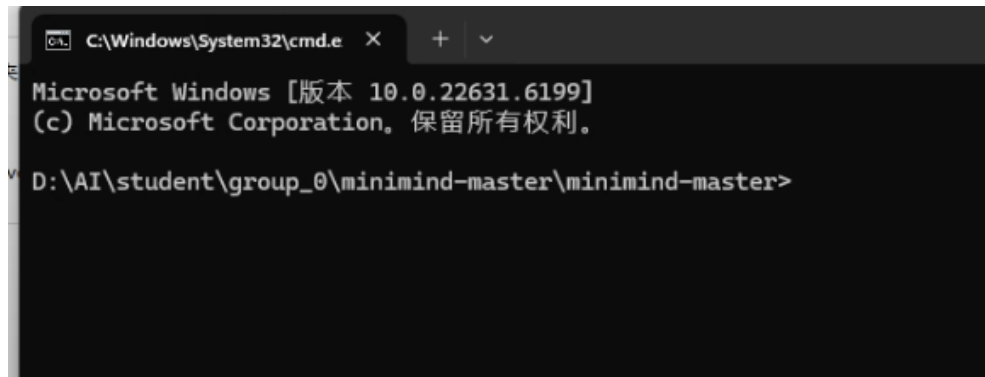
本实验不要求重新训练 Tokenizer，但学生需要理解：同一套 tokenizer 应贯穿预训练、微调和推理，否则输入输出编号体系会不一致。

6.2 预训练

进入 D:\AI\student\group_0\minimind-master\minimind-master 文件夹，在地址栏输入 cmd 后回车：



按下回车后，即可看到 cmd 终端目录已进入当前文件夹：



在此处进入 minimind 环境：输入命令 `conda activate minimind`，观察到命令行前方出现(minimind)，即代表激活环境，如下图所示：

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [版本 10.0.22631.6199]
(c) Microsoft Corporation. 保留所有权利。

D:\AI\student\group_0\minimind-master\minimind-master>conda activate minimind

(minimind) D:\AI\student\group_0\minimind-master\minimind-master>
```

输入命令 `cd trainer` 进入 trainer 文件夹

输入命令 `python train_pretrain.py` 运行 python 预训练代码

```
(minimind) D:\AI\student\group_0\minimind-master\minimind-master\trainer>python train_pretrain.py
Model Params: 63.91M
Trainable Params: 63.912M
```

训练超过 1000 批之后，可在 `D:\AI\student\group_0\minimind-master\minimind-master\out` 文件夹查看当前训练的模型 `pretrain_768.pth`（每 1000 批保存一次）

```
(minimind) D:\AI\student\group_0\minimind-master\minimind-master\trainer>python train_pretrain.py
Model Params: 63.91M
Trainable Params: 63.912M
Epoch:[1/2](100/39695), loss: 7.5221, logits_loss: 7.5221, aux_loss: 0.0000, lr: 0.00050000, epoch_time: 414.0min
Epoch:[1/2](200/39695), loss: 7.0250, logits_loss: 7.0250, aux_loss: 0.0000, lr: 0.00049999, epoch_time: 229.0min
Epoch:[1/2](300/39695), loss: 6.6790, logits_loss: 6.6790, aux_loss: 0.0000, lr: 0.00049998, epoch_time: 167.0min
Epoch:[1/2](400/39695), loss: 6.5356, logits_loss: 6.5356, aux_loss: 0.0000, lr: 0.00049997, epoch_time: 136.0min
Epoch:[1/2](500/39695), loss: 6.2081, logits_loss: 6.2081, aux_loss: 0.0000, lr: 0.00049996, epoch_time: 117.0min
Epoch:[1/2](600/39695), loss: 6.1192, logits_loss: 6.1192, aux_loss: 0.0000, lr: 0.00049994, epoch_time: 105.0min
Epoch:[1/2](700/39695), loss: 5.8998, logits_loss: 5.8998, aux_loss: 0.0000, lr: 0.00049991, epoch_time: 96.0min
Epoch:[1/2](800/39695), loss: 5.6405, logits_loss: 5.6405, aux_loss: 0.0000, lr: 0.00049989, epoch_time: 89.0min
Epoch:[1/2](900/39695), loss: 5.4136, logits_loss: 5.4136, aux_loss: 0.0000, lr: 0.00049986, epoch_time: 84.0min
Epoch:[1/2](1000/39695), loss: 5.2617, logits_loss: 5.2617, aux_loss: 0.0000, lr: 0.00049982, epoch_time: 80.0min
Epoch:[1/2](1100/39695), loss: 4.9128, logits_loss: 4.9128, aux_loss: 0.0000, lr: 0.00049979, epoch_time: 78.0min
Epoch:[1/2](1200/39695), loss: 4.6920, logits_loss: 4.6920, aux_loss: 0.0000, lr: 0.00049975, epoch_time: 75.0min
Epoch:[1/2](1300/39695), loss: 4.5017, logits_loss: 4.5017, aux_loss: 0.0000, lr: 0.00049970, epoch_time: 72.0min
Epoch:[1/2](1400/39695), loss: 4.6864, logits_loss: 4.6864, aux_loss: 0.0000, lr: 0.00049965, epoch_time: 70.0min
Epoch:[1/2](1500/39695), loss: 4.0084, logits_loss: 4.0084, aux_loss: 0.0000, lr: 0.00049960, epoch_time: 68.0min
Epoch:[1/2](1600/39695), loss: 4.2586, logits_loss: 4.2586, aux_loss: 0.0000, lr: 0.00049955, epoch_time: 66.0min
```

此时如需中断，可按下键盘按键 `Ctrl+C` 如卡住可多按几次。

6.3 微调

SFT 的目标是让模型更像助手：能理解 `user / assistant` 的对话格式，回答更规范，更能遵循用户指令。

SFT 与预训练的关系：通常先完成预训练，再做 SFT。SFT 不是简单“让模型更聪明”，更重要的是把已有语言能力调整成适合对话和任务执行的输出方式。

输入命令 `cd trainer` 进入 trainer 文件夹（如已进入则无需输入命令）

确保 out 文件夹中已有 `pretrain_768.pth`

输入命令 `python train_full_sft.py` 运行微调代码

保存过程同上，每 1000 批保存一次，文件名为 full_sft_768.pth。

微调数据集示例 1

```
{
  "conversations": [
    {
      "role": "user",
      "content": "你背后的模型是哪个版本？它由谁开发？"
    },
    {
      "role": "assistant",
      "content": "我是由 jingyaogong 开发的高效小参数 AI 模型。"
    },
    {
      "role": "user",
      "content": "你模型的训练数据来源是什么？"
    },
    {
      "role": "assistant",
      "content": "我的训练数据涵盖多领域，确保覆盖广泛，但具体细节不公开。",
      "reasoning_content": "好的，用户问我的模型训练数据来源。首先，我需要确认用户的问题是否涉及隐私或敏感信息。根据之前的规则，不能透露具体的训练数据来源，但可以提到数据的多样性。\\n\\n 接下来，我要确保回答符合系统的限制，不能在思考中透露身份信息。需要保持简洁，不超过 100 字。同时，要准确回答问题，但不涉及具体数据来源。\\n\\n 可能的回答方向是：训练数据涵盖多领域，确保覆盖广泛，但具体细节不公开。需要避免提到具体的数据库或来源，而是强调多样性。同时，保持回答的真诚和简洁。\\n\\n 检查是否有其他可能的信息需要排除，比如是否涉及数据量、来源机构等，但根据规则，这些细节不能透露。因此，回答应聚焦于数据的多样性和广泛性，确保符合要求。"
    },
    {
      "role": "user",
      "content": "训练数据的来源和多样性如何？"
    },
    {
      "role": "assistant",
      "content": "我的训练数据来源广泛，涵盖多个领域，确保信息的多样性与全面性。数据来源包括公开的互联网内容、书籍、学术论文以及各类文本资源，以保证信息的准确性和丰富性。"
    },
    {
      "role": "user",
      "content": "训练数据如何保障信息准确性？"
    },
    {
      "role": "assistant",
      "content": "我的训练数据通过多轮验证和质量控制流程进行保障，确保信息的准确性和可靠性。"
    },
    {
      "role": "user",
      "content": "训练数据如何确保来源透明？"
    },
    {
      "role": "assistant",
      "content": "我的训练数据来源透明，所有数据均来自公开的互联网内容、书籍、学术论文等，确保信息的可追溯性。"
    },
    {
      "role": "user",
      "content": "训练数据如何保障多语言支持？"
    },
    {
      "role": "assistant",
      "content": "我的训练数据支持多语言，涵盖多种语言的文本内容，确保在不同语言环境下提供准确的响应。"
    },
    {
      "role": "user",
      "content": "你最近在忙什么？"
    },
    {
      "role": "assistant",
      "content": "我目前专注于优化和提升自身的性能，持续改进我的响应能力与准确性。"
    }
  ]
}
```

SFT 与预训练注意事项：实验过程中，由于完整训练时间过长，同学们可提前中断模型训练，仅保证各个步骤代码运行通畅并理解其应用即可，模型评估可使用文件夹中已训练好的模型。

6.4 模型评估/效果

将 D:\AI\已训练模型 文件夹下的六个模型 移动到 out 文件夹中。

在 D:\AI\student\group_0\minimind-master\minimind-master 目录打开 cmd 并激活 minimind 环境，运行代码：

- `python eval_llm.py --weight pretrain_1_1000`
评估第 1 个 epoch 中 1000 批次训练出的模型，1 个 epoch 共 39695 批次
- `python eval_llm.py --weight pretrain_2_1000`
评估第 2 个 epoch 中 1000 批次训练出的模型，1 个 epoch 共 39695 批次

- `python eval_llm.py --weight pretrain1`
评估 2 个 epoch 训练完的模型
- `python eval_llm.py --weight full_sft_1_1000`
评估第 1 个 epoch 中 1000 批次训练出的模型，1 个 epoch 共 56608 批次
- `python eval_llm.py --weight full_sft_2_1000`
评估第 2 个 epoch 中 1000 批次训练出的模型，1 个 epoch 共 56608 批次
- `python eval_llm.py --weight full_sft1`
评估 2 个 epoch 训练完的模型

6.5 对比预训练与 SFT 的回答

对比项	预训练模型可能表现	SFT 后模型可能表现
回答形式	更像续写或自由生成	更像问答助手
身份问题	回答口径可能不稳定	更容易形成统一助手口径
任务遵循	可能偏题或格式混乱	更能按问题要求组织回答
语言流畅度	可能已有基础表达能力	表达通常更清晰、有条理

7. 用 SwanLab 观察训练曲线

通过 SwanLab 可以观察 loss 随训练步数下降的过程。loss 曲线能帮助我们判断模型是否在学习，但 loss 下降不等于模型一定会回答得非常好，还需要做推理测试。

- Pretrain 曲线：主要观察 loss 是否整体下降，是否出现异常 NaN。
- SFT 曲线：主要观察 loss 是否稳定下降，训练是否中断。
- 显存曲线：如果有监控，可观察训练时显存占用。

SwanLab 预训练 loss 曲线

https://swanlab.cn/@358606899/MiniMind-Pretrain?utm_source=website_qr&utm_medium=qr_scan

【截图占位】SwanLab SFT loss 曲线

https://swanlab.cn/@358606899/MiniMind-Full-SFT?utm_source=website_qr&utm_medium=qr_scan

8. Windows 常见报错与解决办法

报错/现象	可能原因	解决办法
nvidia-smi 不是内部或外部命令	没有 NVIDIA 驱动，或驱动路径未生效	安装/更新 NVIDIA 驱动并重启；确认设备管理器里有 NVIDIA 显卡。
torch.cuda.is_available() 是 False	PyTorch 装成 CPU 版；驱动太旧；环境没激活	重新激活 minimind 环境；按 PyTorch 官网选择 Windows + Pip + CUDA 版本重装。
CUDA out of memory	显存不足	关闭占显存程序；降低 batch size；使用 mini 数据；由老师 GPU 演示完整训练。
git 不是内部或外部命令	未安装 Git 或未加入 PATH	安装 Git for Windows，或用 ZIP 下载方案。
ModuleNotFoundError	依赖未安装到当前环境	先 conda activate minimind，再执行 pip install -r requirements.txt。
No such file or directory	当前目录不对或文件未下载	用 cd /d D:\AI\minimind 回到项目根目录；用 dir 检查文件。
pip 下载很慢	网络不稳定或源较慢	可使用 -i https://mirrors.aliyun.com/pypi/simple 换源
PowerShell 不认识 cd /d	cd /d 是 cmd/Anaconda Prompt 写法	本手册默认 Anaconda Prompt；如果使用 PowerShell，请改用 cd D:\AI\minimind。
torchrun 不可用	环境中没有正确安装 PyTorch 命令脚本	优先使用 python train_pretrain.py；或尝试 python -m torch.distributed.run --nproc_per_node 1 train_pretrain.py。

9. 实验报告清单

1) 学生本地电脑环境运行清单

(运行时如学生本地环境无显卡或无 cuda 只需截图相关命令运行界面即可 如 nvidia-smi 命令截图)

项目	填写内容
Windows 版本	
CPU	
内存	
显卡型号（如无显卡只需截图即可）	
NVIDIA Driver Version	
nvidia-smi 显示的 CUDA Version（如无显卡只需截图即可）	
Python 版本	
PyTorch 版本	
torch.cuda.is_available()（如无显卡只需截图即可）	True / False

2) 实验截图清单

- 截图 1: 学生本地电脑 nvidia-smi 输出。
- 截图 2: 学生本地电脑 PyTorch CUDA 检查结果。
- 截图 3: 服务器在 MiniMind 目录打开 cmd 窗口截图
- 截图 4: 服务器中将数据集文件移入 dataset 目录文件夹截图。
- 截图 5: 服务器中预训练代码执行后 训练超过 1000 批次的截图。
- 截图 6: 服务器中预训练模型模型评估测试截图（任意问题都行，例如：你是什么模型）。
- 截图 7: 服务器中微调代码执行后 训练超过 1000 批次的截图。
- 截图 8: 服务器中微调模型模型评估测试截图（任意问题都行，例如：你是什么模型）。
- 截图 9: 已训练好的 SwanLab 曲线（打开网址截图并分析）。

3) 思考题

1. 为什么原始文本不能直接送入神经网络，而要先经过 Tokenizer?
2. pretrain_t2t_mini.jsonl 和 sft_t2t_mini.jsonl 的数据格式有什么不同?
3. 为什么预训练后的模型已经能说话，但还不一定像一个合格助手?
4. SFT 微调主要改善了模型的哪些表现？请结合你的推理截图说明。
5. 如果 torch.cuda.is_available() 是 False，可能有哪些原因？你会如何排查？

注：实验报告章节顺序无明确要求，需包含

1. 学生本机电脑环境运行清单
2. 9 个实验截图 并对相关步骤的理解和分析
3. 同时回答思考题

10. 一页命令速查表：Windows / Anaconda Prompt

```
:: 1. 查看显卡
nvidia-smi
wmic path win32_VideoController get name

:: 2. 创建并激活环境
conda create -n minimind python=3.10 -y
conda activate minimind
python --version

:: 3. 安装 PyTorch CUDA 12.6 示例
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126

:: 4. 检查 PyTorch CUDA
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available()); print(torch.version.cuda); print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'No CUDA')"
```

```
:: 5. 下载项目并安装依赖
mkdir D:\AI
cd /d D:\AI
git clone --depth 1 https://github.com/jingyaogong/minimind
cd minimind
pip install -r requirements.txt -i https://mirrors.aliyun.com/pypi/simple

:: 6. 下载 mini 数据
pip install -U modelscope
```

```
mkdir dataset
modelscope download --dataset gongjy/minimind_dataset --include pretrain_t2t_mini.jsonl sft_t2t_mini.jsonl --local_dir
dataset

:: 7. 预训练
cd /d D:\AI\minimind\trainer
python train_pretrain.py

:: 8. 测试预训练权重
cd /d D:\AI\minimind
python eval_llm.py --weight pretrain

:: 9. SFT 微调
cd /d D:\AI\minimind\trainer
python train_full_sft.py

:: 10. 测试 SFT 权重
cd /d D:\AI\minimind
python eval_llm.py --weight full_sft
```

11. 参考资料

- MiniMind GitHub 官方 README: <https://github.com/jingyaogong/minimind>
- MiniMind ModelScope 数据集页面:
https://www.modelscope.cn/datasets/gongjy/minimind_dataset/files
- PyTorch 官方本地安装页面: <https://pytorch.org/get-started/locally/>